AUTOFORGE: A CHECKPOINTED, HUMAN-IN-THE-LOOP MULTI- AGENT SDLC PIPELINE

INDIVIDUAL THESIS

OBERATHNA R.D.T.D

IT22102478

Individual component: Security Analysis and Quality Assurance

Group ID: R26-SE-033

Thesis submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honors) in Information Technology

Specializations in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Oberathna R.D.T.D

Student ID: IT22102478

Signature: _____  _____ Date: ____05/10/2026_____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Prof. Nuwan Kodagoda		
Co-supervisor: Mr. Eishan Weerasinghe		10/10/2026

ABSTRACT

Code written by a language model can build, start and still be unsafe or wrong, and a reviewer cannot read every line. This thesis covers the Security and Quality Assurance (QA) agents of AutoForge, a seven-agent, human-gated pipeline for e-commerce web development. The Security Agent runs three deterministic layers over the generated feature: a pattern scanner that walks the TypeScript syntax tree with seven CWE-tagged rules, a secret scanner, and npm audit in a sandbox. An optional AI layer either reviews the findings summary or reads the source directly. Findings are mapped to Critical, Moderate and Warning tiers, and the report waits for a person who can proceed or send it to the Coder Agent as a revision, after which the feature is scanned again. The QA Agent finds testable modules and Route Handlers, generates unit, integration and regression tests, the last from the approved acceptance criteria, runs them with Jest in the sandbox, and explains failures. I evaluated the agents with automated tests, the team's-controlled vulnerability experiment (precision 87.5%, recall 77.8%), the three-feature case study and 22 security and 8 QA reports stored for two features. The deterministic layers were consistent and caught a hardcoded database credential and unsafe HTML rendering. The AI deep scan found serious issues they missed, such as NoSQL injection and a weak password-hashing cost, but its count varied from three to eight across repeated scans whose deterministic results did not change, and it produced clear false positives. In QA, every failure on one feature came from a generated test that forgot to import the handler, not from the application. Generated checks need the same scrutiny as generated code, and a person must still judge which findings and failures are real.

Keywords: software security, static analysis, test generation, human-in-the-loop, multi-agent systems

ACKNOWLEDGEMENTS

I am thankful to my supervisor, Prof. Nuwan Kodagoda, and my co-supervisor, Mr. Eishan Weerasinghe, for the assistance they provided throughout the project, especially for the emphasis they placed on the fact that a clean scan does not mean that the application is safe and thus set the stage for carrying out a controlled vulnerability experiment.

My appreciation also goes to my colleagues, Santhuka D.N.M.D., Sansala T.G.B.D. and Dissanayake T.C., whose work my agents test, making every finding and each unsuccessful test an occasion for us to converse regarding whether the issue lay in the code or my checks.

Due thanks also to the academic staff at the Software Engineering Department of Sri Lanka Institute of Information Technology as well as to the members of the panel at the progress presentations who provided me with their feedback. Last but not least, I would like to express my gratitude to my family and friends for supporting me throughout the completion of this project.

Table of Contents

DECLARATION AND SUPERVISOR CERTIFICATION ii
ABSTRACT iii
ACKNOWLEDGEMENTS iv
LIST OF FIGURES ix
LIST OF TABLES x
LIST OF ABBREVIATIONS xi
1.1 Overall Project Context 1
1.2 Individual Component Context 2
1.3 Problem Statement 2
1.4 Research Gap 3
1.5 Aim and Objectives 3
1.6 Research Questions 4
1.7 Scope, Limitations and Assumptions 4
1.8 Individual Contribution 4
1.9 Thesis Structure 5
2 LITERATURE REVIEW AND RELATED WORK 6
2.1 Review Method and Scope 6
2.2 Domain and Theoretical Foundations 6
2.2.1 Static analysis and weakness categories 6
2.2.2 Secrets and dependencies 6
2.2.3 LLMs for security review and testing 7
2.2.4 Evaluating detection 7
2.2.5 Human oversight and science 7
2.3 Related Research 7
2.4 Existing Systems, Products and Techniques 8

2.5 Gap Synthesis and Conceptual Framework 9
3 Research Methodology 10
3.1 Research Design 10
3.2 Data, participants and Study Materials 10
3.3 Experimental and Study Procedure 11
3.4 Measures and Evaluation Metrics 12
3.5 Data Analysis 12
3.6 Ethics, Privacy, Security and Responsible AI 12
3.7 Reproducibility and Research Artefacts 13
4 PRODUCT AND SOLUTION DESIGN 14
4.1 Requirements 14
4.2 Component Architecture within the Complete Solution 15
4.3 Design Alternatives and Decisions 16
4.4 Detailed Design 17
4.4.1 Deterministic security layers 17
4.4.2 Model Layers 18
4.4.3 Severity tiers and the gate decisions 19
4.4.4 Reports, decisions, and the fix loop 19
4.4.5 Test discovery and generation 19
4.4.6 Execution, matching and analysis 20
4.5 Standards and Professional Practice 21
5 IMPLEMENTATIONS 22
5.1 Development Environment and Technology Stack 22
5.2 Implementation of the Core Modules 22
5.2.1 Security scanners 23
5.2.2 Model layers and reports 24

5.2.3 QA discovery, generation and execution 24
5.2.4 Interface support 24
5.3 Integration with the Complete Solution 25
5.4 Testing and Quality assurance 25
5.5 Deployment and Operation 25
6 RESULTS AND EVALUATION 27
6.1 Component-Level Test Results 27
6.2 Experimental Results 28
6.2.1 Detection of Known vulnerabilities (RQ1) 28
6.2.2 Consistency between scans (RQ2) 29
6.2.3 Evidence from generated tests (RQ3) 30
6.2.4 The fix loop (RQ4) 31
6.3 Stakeholder and User Validation 32
6.4 Integration and Performance Results 32
6.5 Summary of Findings 33
7 DISCUSIONS 34
7.1 Interpretation of the Results 34
7.2 Comparison with Related Work 35
7.3 Implication for Practice 35
7.4 Threats to Validity 35
7.5 Limitations 36
8 COMMERCIALISATION AND SUSTAINABILITY 37
8.1 Value of the Component 37
8.2 Market competitive position 38
8.3 Business Model and Feasibility 39
8.4 Sustainability and Maintenance 41

8.5 Contribution to the Sustainability Development Goals 41
9 CONCLUSION AND FUTURE DIRECTIONS 42
9.1 Conclusion 42
9.2 Achievement of Objectives 42
9.3 Contribution 43
9.4 Future Directions 43
REFERENCES 44
GLOSSARY 46
APPENDICES 48
Appendix A: Supporting Technical Material 48
A.1 Deterministic security rules 48
A.2 Test catalogue 49
Appendix B: Disclosure of AI Tool Use 49

LIST OF FIGURES

Figure 1.1 The AutoForge System Diagram 1
Figure 2.1: Conceptual framework for the security and QA component 9
Figure 4.1: The component's place in the complete AutoForge architecture 16
Figure 4.2: Security Agent pipeline 18
Figure 4.3: QA Agent pipeline 20
Figure 6.1: Findings per security report for the Login and Signup feature, by producing layer .. 30

LIST OF TABLES

Table 2.1:Checking-stage properties of related systems 8
Table 3.1: Study materials 10
Table 3.2: Vulnerabilities seeded into the Login and Signup feature 11
Table 3.3: Measures used for each research question 12
Table 3.4: Configuration parameters of the component 13
Table 4.1: Functional requirements of the component 14
Table 4.2: Non-functional requirements of the component 15
Table 4.3: Main design decisions 16
Table 4.4: Severity mapping and gate decision 19
Table 5.1: Technology stack used by the component 22
Table 5.2: Main modules of the component 22
Table 5.3: Automated tests for the component 25
Table 6.1: Key behaviors confirmed by automated tests 27
Table 6.2: Detection of the seeded vulnerabilities by layer 28
Table 6.3: Outcome of all test cases in the stored QA reports 30
Table 6.4: Execution performance of the full pipeline 32
Table 6.5: Summary of findings by research question 33
Table 8.1: Target users of the component 37
Table A.1: Pattern and secret rules 48
Table A.2: Test files for the component 49
Table B.1: Use of generative AI tools 50

LIST OF ABBREVIATIONS

Abbreviation	Description
AC	Acceptance Criterion
AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
CRUD	Create, Read, Update and Delete
CWE	Common Weakness Enumeration
FN	False Negative
FP	False Positive
JSON	JavaScript Object Notation
LLM	Large Language Model
npm	Node Package Manager
QA	Quality Assurance
RQ	Research Question
SDG	Sustainable Development Goal
SDLC	Software Development Life Cycle
SRS	Software Requirements Specification
TP	True Positive
UI	User Interface
XSS	Cross-Site Scripting

1 INTRODUCTION

1.1 Overall Project Context

The project AutoForge is an online shopping application created by our team during our last year of research. It works through several steps, similar to the workflow of software engineering companies. The first step is the phase where the Requirement Agent collects the Software Requirements Specification (SRS). Next comes the Domain Agent, who adds more features to the SRS. The next step is the Architectural Stage, where a technical plan is created by the Architecture Agent. After that, the user interfaces are created by the UI/UX Agent. Once everything is done, coding is done by the Coder Agent, and finally, the Security Agent and QA Agent measure whether the work is completed correctly.

The project is motivated by the observation that an LLM can write working code on request, but a tool that goes straight from a prompt to code gives the user nothing to check, and no way to tell whether a bad result came from the requirement, the design or the code. Multi-agent frameworks such as ChatDev [1] and MetaGPT [2] separate these roles, but their checking is done by agents talking to each other. Figure 1.1 shows the AutoForge pipeline from the paper.

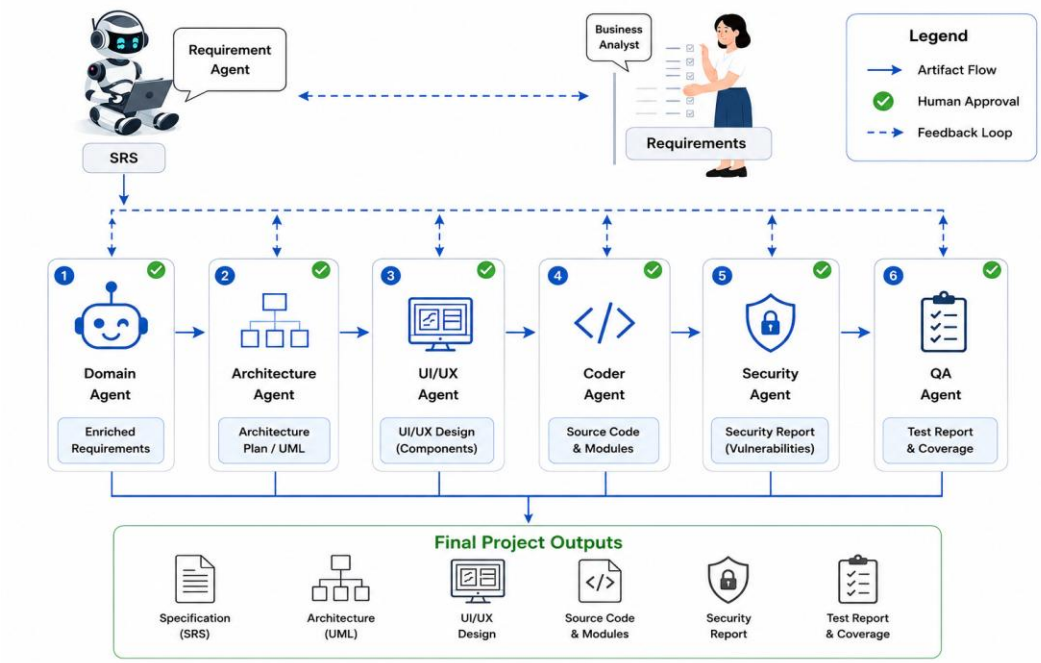


Figure 1.1: The AutoForge System Diagram

The project was divided into four components, one per team member. My component is the last two stages of the pipeline: the Security Agent, which scans the generated feature for vulnerabilities, and the QA Agent, which writes and runs tests against it. The other components cover requirements and orchestration, architecture and UI/UX design, and code generation. The team's common report describes the integrated system; this thesis describes my component in detail.

1.2 Individual Component Context

By the time a feature reaches my stages, it has been specified, designed, coded and verified to build and start. That verification says the code runs; it says nothing about whether the code is safe or whether it does what the acceptance criteria require. Those are the two questions my agents answer, and they matter particularly in e-commerce, where features handle passwords, personal details and orders.

The two agents share a principle: produce evidence, not opinions. The Security Agent's main findings come from deterministic analysis of the real code and its dependencies, each tied to a file, a line and a Common Weakness Enumeration (CWE) entry. The QA Agent's results come from tests executed in a sandbox, each with its real outcome and failure message. An LLM helps in both, by reviewing findings, reading source for issues rules cannot express, writing tests and explaining failures, but its contribution is labelled and kept separate from the deterministic evidence.

Both agents also close the loop with the Coder Agent. A reviewer can send a security report or the failing tests back as a revision, and the feature is then checked again. This makes my stages the place where problems introduced anywhere earlier in the pipeline are most likely to surface.

1.3 Problem Statement

Security checking and testing of LLM-generated code face three problems. First, the obvious tool for the job, another LLM, is not reliable on its own: it can miss real vulnerabilities, report on ones that are not there, and give a different answer each time it is asked. Second, a clean result is easy to misread. A scan with no findings, or a small test suite that passes, does not show that the code is secure or correct, only that these checks find nothing. Third, generated tests are themselves generated code: when one fails, the fault may lie in the application or in the test.

The problem this thesis addresses is therefore: how can the security and QA stages of an LLM-based pipeline produce evidence about generated code that a person can trust and act on, while using LLMs where they help and being honest about what the evidence does not show?

1.4 Research Gap

Studies on the efficacy of LLMs in security Zhu et al. [3] and software testing Wang et al. [4] indicate that while LLMs show great promise, they fail to be reliable when used alone; Zhu et al. recommended the use of LLMs with conventional static analysis. Coding agents like SWE-agent [5] and OpenHands [6] are tested based on whether currently available tests work or failure logs, while AgentCoder [7] generates tests to lead its own coding process. But these approaches fail to treat the results of security scanning or testing as artifacts that can be analyzed by an individual at the preliminary stage of modifications. The multi-agent systems ChatDev [1] and MetaGPT [2] have roles in testing, but their verification is done mainly through communication between agents. One of the limitations is that security and QA stages provide proof, including links to code and well-known categories of weaknesses, executed tests, deterministic results vs. model-generated ones, etc.

1.5 Aim and Objectives

This project aims to create the AutoForge Security Agent and AutoForge QA Agent, which will allow every feature produced by AutoForge to be automatically scanned for vulnerabilities and tested for quality, and generate structured and timestamped evidence for the human reviewer before the code generation process resumes.

1. O1: Create the Security Agent which will provide deterministic vulnerability scanning and LLM review and report the results for further analysis by the humans.

2. O2: Create the QA Agent which will help identify the code generated by the system to be tested, generate tests with LLM and fallback algorithms, run the tests in the sandbox, and report the test results for analysis by humans.

3. O3: Connect Security and QA Agents to the human review process enabling feedback from the tests and vulnerabilities found to be used as input for the next cycle of code generation and verification.

4. O4: Determine the practical usage and accuracy of the Security & QA Agents via the tests and reviews conducted.

1.6 Research Questions

1. RQ1: How accurately does the layered Security Agent detect known vulnerabilities in generated code?
2. RQ2: How do the deterministic layers and the AI layer differ in what they find and in how consistent they are?
3. RQ3: Do the generated tests provide reliable evidence about the generated application?
4. RQ4: Does putting a person between the reports and the next step, with a loop back to the Coder Agent, lead to real fixes?

1.7 Scope, Limitations and Assumptions

The Coder Agent produces the Next.js App Router application in TypeScript and MongoDB to allow the agents to focus on their work. The Security Agent utilizes seven different code structures, and five different forms of hardcoded secrets, relying on the npm advisory database to help its testing. However, it has no full static analyzer features and does not assess an active application. The QA Agent uses several modules and Route Handlers without requiring browser action as pages and React components are not included in the scope of tests.

The research was not done on a large scale, with only a few system vulnerabilities revealed in the experiments with the case study disclosing 3 system characteristics, while two other system features were made known by a report review. The outcomes of the research indicate the performance of the agents, showing their advantages and their drawbacks at the same time, but fail to demonstrate effectiveness in the general sense.

It is assumed that the feature has already passed the examination by the Coder Agent and that the sandbox environment is established, while the report is reviewed by a knowledgeable person who can make sense of the information demonstrated within the report.

1.8 Individual Contribution

Within the team, my responsibility was the security and QA stages. My contributions are:

- Creation of the Security Agent, which includes development of the pattern scanner based on TypeScript compiler's syntax tree, secret scanner, dependency scanner, summary review layer, and deep source scan using AI.
- Implementation of the stages of security severity and gate decision mechanism, along with security report generation in JSON, Markdown, and PDF formats, identification of accepted risk levels, and comparison of different security report versions.
- Development of the mechanisms for test discovery, test generation (unit, integration, and regression testing) of the QA Agent, as well as fallback strategies, Jest test executor, test result matching, and root cause analysis.
- Inclusion of references to security and QA reports in the Coder Agent and the chat interface allowing users to ask questions about the generated reports.
- Creation of automated tests for the Security and QA modules, conducting experiments on detection of controlled vulnerabilities, and supporting the team in evaluation and research.

1.9 Thesis Structure

Chapter 2 reviews related work and builds a conceptual framework. Chapter 3 describes the method. Chapter 4 presents the design, and Chapter 5 covers implementation, integration and testing. Chapter 6 reports the results, including the stored reports, and Chapter 7 discusses them. Chapter 8 considers commercialization and sustainability, and Chapter 9 concludes.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review began with the discussion of two broad surveys that served as the basis for my team's paper: that by Hou et al. [8] on LLMs for software engineering and that by He, on LLMs for multi-agent systems. After that, I moved to the two surveys that had the closest linkage to my component: that of Zhu et al. [3] on LLMs and software security and Wang et al. [4] on LLMs and software testing. From these surveys, I followed through the systems and approaches most closely related to my concerns regarding security analysis and testing of generated code. The aim of this review was to position the Security and QA components in the context of existing studies rather than to give a comprehensive review of the entire field.

2.2 Domain and Theoretical Foundations

2.2.1 Static analysis and weakness categories

Static analysis examines code without running it. The simplest form searches text for dangerous patterns, but text search cannot tell a call from a comment or a string that happens to contain the same word. Parsing the code into an abstract syntax tree (AST) solves this: a call to eval is a call-expression node with that name, and nothing else matches. The Common Weakness Enumeration (CWE) gives each kind of weakness a standard identifier, such as CWE-79 for cross-site scripting or CWE-798 for hardcoded credentials, which lets a finding be understood and looked up independently of the tool that produced it.

2.2.2 Secrets and dependencies

Two classes of security problem need no code analysis at all. Credentials written into source files, such as API keys or database connection strings with passwords, can be found by pattern matching over every text file. Known vulnerabilities in third-party packages are published in advisory databases, and npm audit compares a project's locked dependency versions against them. Both are fast and deterministic, and both catch problems an LLM coding agent is prone to hardcoding a value to make something work and adding packages without checking them.

2.2.3 LLMs for security review and testing

Zhu et al. [3] survey how LLMs are used for vulnerability detection and repair and conclude that, while LLMs can recognize issues that rules do not encode, their results vary and should be combined with static analysis rather than replace it. Wang et al. [4] survey LLM-based testing and report wide use for generating unit tests, with recurring problems: tests that do not compile, tests with wrong assertions, and low coverage of real behavior. Both points shape my design directly.

2.2.4 Evaluating detection

A security scanner is measured by comparing its findings with the vulnerabilities actually present. A correct finding is a true positive, a reported issue that is not real is a false positive, and a real issue that is missed is a false negative. Precision, the share of findings that are real, and recall, the share of real issues found, follow from these. Measuring recall requires knowing what is present, which is why vulnerabilities are injected deliberately into a test version of the code; a clean scan of unknown code says nothing about recall.

2.2.5 Human oversight and science

Treude and Lo [9] note that verifiability and human-agent cooperation are major issues when it comes to using large language models in multi-agent systems, while Takerngsaksiri et al. [10] indicate that engineers who work with agents appreciate the possibility of scrutinizing their plans and code. In response to these needs, this work creates a security and quality assurance artifact that enables verification of its results by humans. The artifact is designed and tested using the methodology suggested by Hevner et al. [11], which makes it clear that artifact evaluation is part of knowledge generation in terms of its design, capacity, and failure.

2.3 Related Research

Most coding-agent research evaluates correctness through tests. SWE-bench [12] runs a repository's own tests after an agent's change, and agents such as SWE-agent [5], OpenHands [6], HyperAgent [13] and Agentless [14] are judged this way; Agentless also generates reproduction tests to select its patches. AgentCoder [7] separates a test designer agent from the programmer agent and runs the tests in the loop. These systems show that executed tests are valuable evidence; they do not address security scanning, and their tests guide the agent rather than inform a human reviewer.

ChatDev [1] and MetaGPT [2] include reviewer and tester roles in their simulated software teams. In ChatDev, testing happens in a dialogue between agents; in MetaGPT [2], a QA engineer role writes and runs tests. In both, the outcome feeds back to other agents rather than producing a report for a person to approve, and security is not a separate stage.

Zhu et al. [3] 's recommendation to combine static analysis with LLM review Zhu et al. [3]is followed directly in the Security Agent. None of the multi-agent systems reviewed above reports security precision and recall for the code it generates; the controlled experiment in Chapter 6 measures both for the Security Agent.

2.4 Existing Systems, Products and Techniques

Table 2.1: Checking-stage properties of related systems

System	Security Checking	Test evidence	Result goes to	Human gate
ChatDev [1]	N/R	Agent-run testing in dialogue	Other agents	N/R
MetaGPT [2]	N/R	QA role writes and runs tests	Other agents	N/R
AgentCoder [7]	N/R	Generated tests in the loop	Programmer agent	N/R
SWE-agent [5]	N/R	Repository tests after the run	Benchmark	N/R
AutoForge Security + QA (this work)	**AST, secrets, npm audit + AI layer**	**Generated and executed unit, integration, regression tests**	**Human reviewer, then Coder Agent**	**Yes (security report)**

In industry, static analysis tools, secret scanners and dependency auditing are standard parts of a continuous integration pipeline, and AI assistants now offer both security review and test generation. My component does not try to replace mature tools; its rule set is small. What it adds is the integration of these techniques into a human-gated agent pipeline, with labelled AI findings, a loop back to the agent that wrote the code, and an honest account of how the AI parts behave.

2.5 Gap Synthesis and Conceptual Framework

The review shows that LLM-based development systems rarely treat security as a separate stage, judge correctness through tests that guide agents rather than inform people, and seldom separate deterministic evidence from model judgement. Figure 2.1 shows the conceptual framework I used: the coded feature, its acceptance criteria and the shared sandbox go through deterministic and model-based mechanisms, controlled by severity tiers, result matching and a human decision, to reports that can drive a fix.

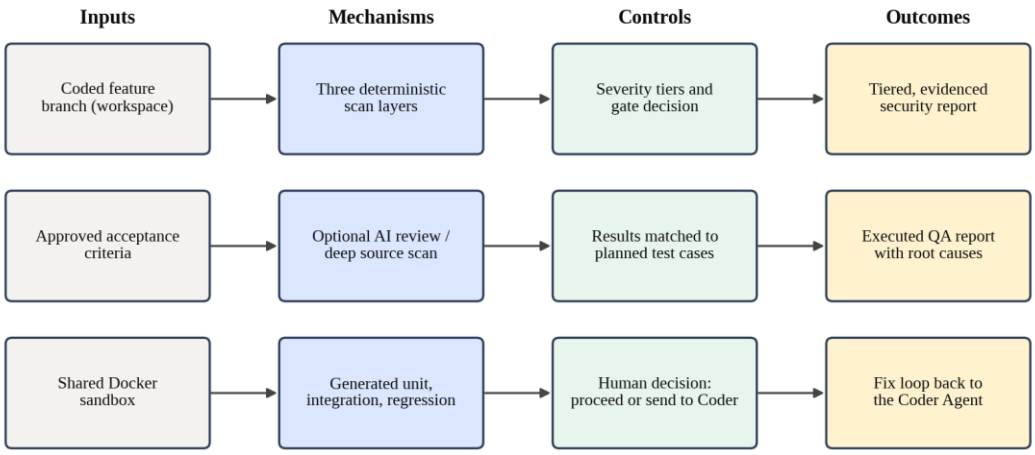


Figure 2.1: Conceptual framework for the security and QA component

3 Research Methodology

3.1 Research Design

The research employed a design-science approach based on the concept introduced by Hevner et al. [11]. Two artefacts, the Security Agent and the QA Agent, were created and tested using the AutoForge system for which they were designed. The evaluation was performed using automated testing of agent's behavior, controlled experiments and analysis of various security and QA reports saved by agents concerning two features developed during the project.

The development process included several build-and-evaluate cycles carried out from July to August 2026. Different design modifications could be implemented because of various failures observed during testing. The failures included inability to activate pattern matching rules due to specific way the syntax tree assistant detected method calls, false secret findings due to the content of local environment file, and inability to correlate the output of the testing program with actual tests. These failures were reported as design discoveries that illustrate the evolution of artefacts in response to limitations encountered.

3.2 Data, participants and Study Materials

Table 3.1: Study materials

Material	**Source**	**Used for**
Controlled vulnerability set	Finodil project, stored artefacts	Precision and recall (RQ1)
Item Listing, Authentication, Shopping Cart	Team case study	Security findings and QA results per feature
Expanded QA run	Retail Store project, stored artefacts	Tests by category
Login and Signup: 19 security and 7 QA reports	Finodil project, stored artefacts	Layer behaviour, consistency, fix loop (RQ2-RQ4)
Add & List Items: 3 security and 1 QA report	Retail Store project, stored artefacts	False positives, test reliability (RQ2, RQ3)

The default model was qwen3-coder through Ollama. No external participants took part, and no personal data was collected. Some stored reports quote credentials that appeared in generated code; these are not reproduced in this thesis.

3.3 Experimental and Study Procedure

Controlled detection experiment. Because the case-study features produced few or no findings, which cannot establish recall, the team injected known vulnerabilities into isolated versions of a generated application, ran the Security Agent, and classified each finding as a true positive, a false positive or, for injected issues not found, a false negative.

Seeded feature. To observe the layers on known vulnerabilities in our own stored data, five vulnerabilities were deliberately introduced into the Login and Signup feature of the Finodil project in a single commit on 22 August 2026, whose message records them as authorised test vulnerabilities for verifying the Security Agent. Table 3.2 lists them. Every security report produced while this code was in place was analysed, and so were the reports after the Coder Agent's fixes.

Table 1.2: Vulnerabilities seeded into the Login and Signup feature

ID	Vulnerability	Location	CWE
V1	MongoDB connection string with a hardcoded credential as a fallback	lib/mongodb.ts	CWE-798
V2	Error message rendered with dangerouslySetInnerHTML	Login and signup page	CWE-79
V3	Client-supplied filter object merged directly into the login query	Login route	CWE-943
V4	bcrypt cost factor of 1 when hashing passwords	Signup route	CWE-916
V5	Plaintext password written to the log on login	Login route	CWE-532

Case study. Each of three features went through all seven stages, and the security findings and QA results were recorded per feature. An expanded QA run then classified tests into unit, regression and integration categories and recorded executed, passed and failed counts.

Stored-report analysis. For the two development features I examined every stored report: for security, each finding's layer, rule, severity, CWE and location, and how the findings changed between versions; for QA, each test's category, outcome, failure message and root cause. Where the same code was scanned more than once, I compared the results to measure consistency.

Component testing. The scanners, severity mapping, deep-scan batching, result matching, Jest result parsing, generation fallbacks and routes were tested against fixtures, with the Node helper, the sandbox and the model replaced by stubs where needed.

3.4 Measures and Evaluation Metrics

Table 3.3: Measures used for each research question

RQ	Measure	How it is obtained
RQ1	True positives, false positives, false negatives, precision, recall	Controlled injection experiment
RQ2	Findings per layer; variation across repeated scans; false positives and mislabelled CWEs	Stored security reports
RQ3	Tests generated, passed, failed, skipped; cause of each failure (application or test)	Stored QA reports; expanded QA run
RQ4	Findings resolved after a revision; revisions requested	Report versions; Coder Agent revision artefacts

Precision is TP / (TP + FP) and recall is TP / (TP + FN).

3.5 Data Analysis

The analysis is descriptive. The detection experiment is small, so precision and recall are reported with their underlying counts. For the stored reports, I report counts and individual cases, and I separate findings and failures that reflect a real problem in the application from those caused by the check itself, because the two lead to opposite actions.

3.6 Ethics, Privacy, Security and Responsible AI

Security tooling handles sensitive material. The secret scanner reports where a credential appears and what kind it is, and the reports are stored with the project's other artefacts; this thesis does not

reproduce any credential. The secret scanner deliberately skips the git ignored local environment files, where the project is meant to keep its real secrets, so that the correct place for a secret is not reported as a leak. Dependency auditing and test execution run inside the shared Docker sandbox, never on the host.

The AI layers are labelled as such in every report, kept separate from deterministic findings, and allowed to fail without affecting them. A reviewer can mark a finding as an accepted risk instead of fixing it, which is recorded rather than hidden. No security report is acted on automatically a person decides whether to proceed or to send it to the Coder Agent.

3.7 Reproducibility and Research Artefacts

Table 3.4: Configuration parameters of the component

Parameter	Value
Default LLM	qwen3-coder via Ollama
Pattern rules / secret rules	7 / 5
Dependency scan	npm audit --json --offline, 60 s timeout
Deep scan batch size	Up to 12,000 characters of source per call
Deep scan concurrency	Up to 3 batches at a time
Test framework	Jest 29 with Babel TypeScript preset
Severity tiers	Critical, Moderate, Warning

All code and tests are in the team's repository, and every report is a numbered artefact in JSON and Markdown, with a PDF on request. The deterministic layers give the same result for the same code; the AI layers and the generated tests do not, which is itself one of the findings of Chapter 6.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements came from the team's pipeline objectives, from what a reviewer needs in order to act on a report, and from problems found in real runs. Table 4.1 lists the functional requirements and Table 4.2 the non-functional ones.

Table 4.1: Functional requirements of the component

ID	Requirement
C4-FR-01	Scan the generated project for dangerous code patterns using a real syntax tree
C4-FR-02	Scan text files for hardcoded secrets, excluding git-ignored local environment files
C4-FR-03	Audit the project's dependencies against published advisories
C4-FR-04	Optionally add model findings, either by reviewing the deterministic findings or by reading the source
C4-FR-05	Map every finding to Critical, Moderate or Warning and derive a pass, review or fail decision
C4-FR-06	Give every finding a location, CWE, root cause and recommendation
C4-FR-07	Let the reviewer proceed or send the findings to the Coder Agent, then re-scan after the fix
C4-FR-08	Let the reviewer mark individual findings as skipped
C4-FR-09	Discover unit and integration test targets, and list untestable files explicitly
C4-FR-10	Generate unit, integration and one regression test per approved acceptance criterion
C4-FR-11	Run the tests in the sandbox and record each test's real result
C4-FR-12	Explain why failed tests failed
C4-FR-13	Answer the reviewer's questions about a report in a chat

Table 4.2: Non-functional requirements of the component

ID	Quality	Requirement
C4-NFR-01	Groundedness	Deterministic findings must not depend on a model; model findings must be labelled
C4-NFR-02	Robustness	A failed or unparseable model response must never fail the scan or the test run
C4-NFR-03	Honesty	What was not checked or not tested must be stated in the report
C4-NFR-04	Safety	Dependency audits and tests must run in the sandbox, not on the host
C4-NFR-05	Consistency	Severity must be expressed in one fixed vocabulary across all layers
C4-NFR-06	Testability	Rule matching and parsing must be testable without Node or a model
C4-NFR-07	Responsiveness	Long scans and test runs must stream progress to the interface

4.2 Component Architecture within the Complete Solution

Figure 4.1 places the component in the complete system. Both agents run after the Coder Agent, against the same project workspace, and both use the shared Docker sandbox for anything that executes code. The Security Agent's findings, and the QA Agent's failing tests, can travel back to the Coder Agent as a revision request, which lets a later stage send work back to an earlier one without starting that stage again from scratch.

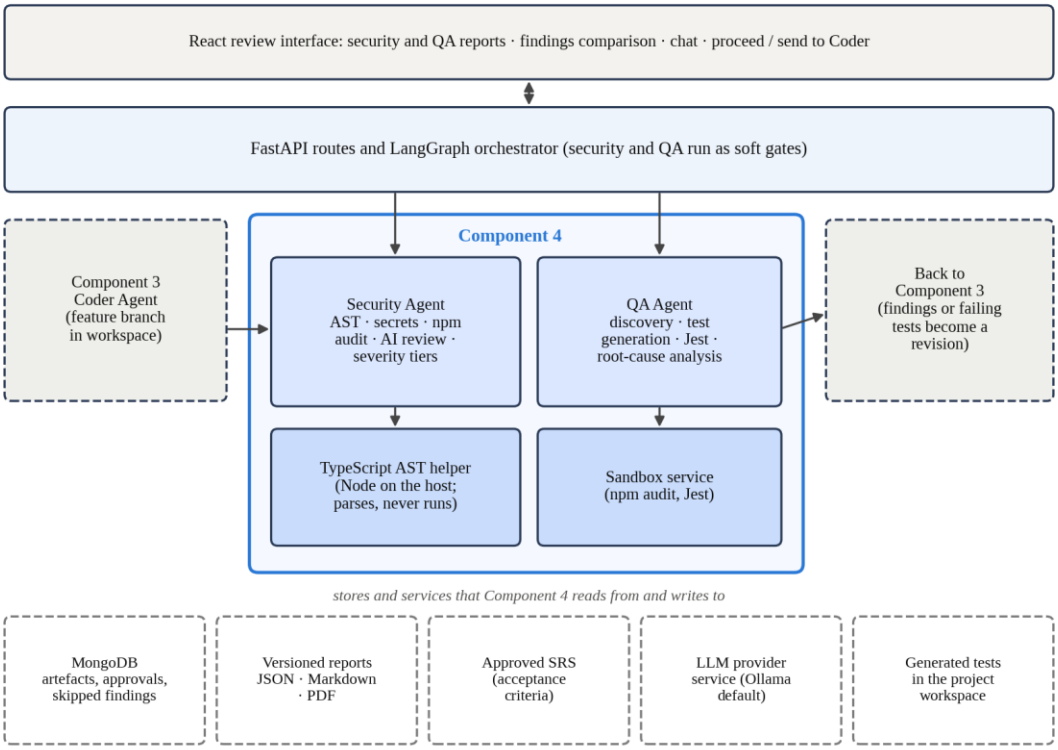


Figure 4.1: The component's place in the complete AutoForge architecture

There are no interruptions to the orchestration graph during the Security and QA stages, which continue running while the pipeline is executed without waiting. However, the reports of these stages are not left without reviews. The report of Security gets saved with the status of pending and is sent for approval by a human reviewer. The decision on the way the pipeline should be carried out is made after the approval. The report provided by QA gets saved as approved and needs to be reviewed by a human, as further discussed in Section 7.5.

4.3 Design Alternatives and Decisions

Table 4.3: Main design decisions

Decision	**Alternative**	**Reason for the choice**
Deterministic layers as the core	Ask an LLM to review the code	Stable, explainable findings; follows the hybrid approach recommended in the literature
AST-based pattern scan	Regular expressions over the source	No false matches in comments or strings; a call is a real call

Two optional model layers	No model layer	Model layers catch logic flaws such as injection that fixed rules miss
Three fixed severity tiers	Pass through each layer's own severity words	One vocabulary for the report, the counts and the interface
Gate decision as advice	Block the pipeline on Critical findings	Heuristic findings can be wrong; a person decides, with a fix loop available
Jest with --json output	Node's built-in test runner	Real per-test structured results without a custom parser
Tests from real source	Tests from signatures only	The model can see what the code actually does and imports
Pages listed as out of scope	Test pages without a browser, or ignore them	Honest about what a DOM-less runner can check

Two developments have occurred in implementation. The first development involves the fact that the security gate now provides suggestions to human reviewers instead of automatically shutting down the security process when serious/important findings arise. This innovation allows the repair loop to work, so that issues can be forwarded to the Coder Agent for fixing and subsequent verification instead of interfering with the whole process.

The second development is related to the fact that the QA Agent employs Jest as a test framework and has extended its testing capabilities by providing not only the testing of modules lib/ and models/.

4.4 Detailed Design

4.4.1 Deterministic security layers

Figure 4.2 shows the security pipeline. The pattern scan parses every JavaScript and TypeScript file with the TypeScript compiler and walks the syntax tree, matching seven rules, listed in Appendix A: eval, the Function constructor, child_process exec, document.write,

dangerouslySetInnerHTML, dynamic property access keyed by request-like names such as sort or field, and Math.random. Each rule carries severity, a CWE, a message, a root-cause template and a recommendation. Two events for the same rule, file and line are reported once.

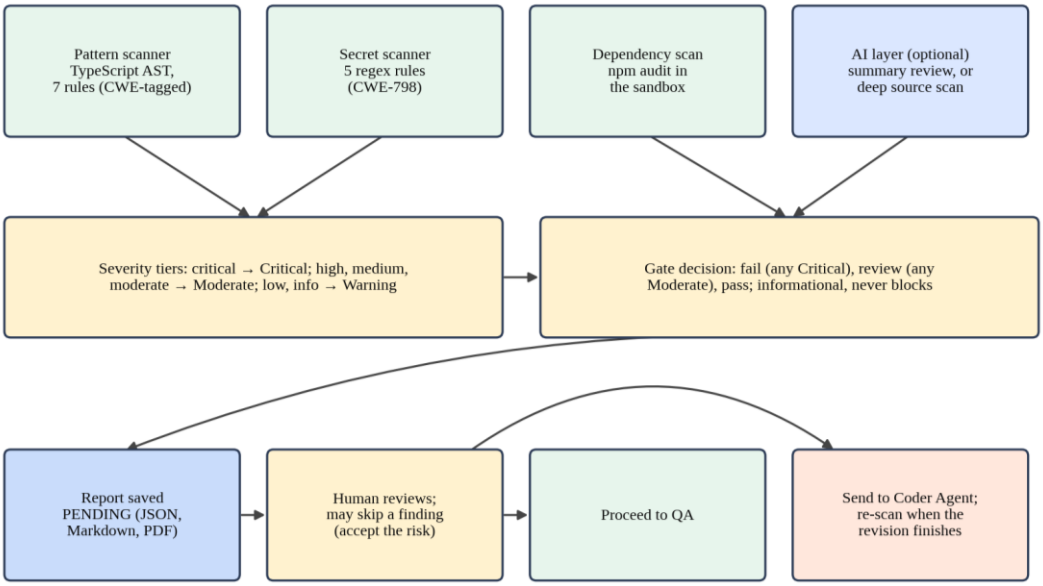


Figure 4.2: Security Agent pipeline

The secret scan uses five regular-expression rules, for AWS access keys, generic API keys and tokens, private key blocks, MongoDB connection strings with an embedded password, and literal JSON Web Tokens, all at critical severity. It reads all text-like files, including. env and .env.example, because secrets leak into configuration as often as into code. It skips files matching. env.local: these are git-ignored and are where AutoForge deliberately stores a project's real database connection, so flagging them was a false positive that could never lead to a leak. The dependency scan runs npm audit with JSON output inside the sandbox and turns each reported package into a finding.

4.4.2 Model Layers

The standard scan adds a review layer: the model is shown a compact summary of the deterministic findings, never the source, and asked to propose further, clearly grounded findings in a fixed JSON shape. The AI deep scan replaces this with a layer that reads the real source. Files are sorted by path, so that related files fall together, grouped into batches under a character limit, and scanned concurrently, with each finding given a sequential identifier. Both layers follow the same rule: a

model call that fails, or returns something that cannot be parsed, contributes no findings and a status note saying why. It never fails the scan, and it never changes the deterministic findings.

4.4.3 Severity tiers and the gate decisions

The layers use two severity vocabularies: the scanners and model layers use critical, high, medium and low, while npm audit uses critical, high, moderate, low and info. A single function maps both onto the three tiers shown in Table 4.4, and every count, badge and decision reads the tier through that function. An unknown severity word, for example one a model invents, maps to Warning rather than causing an error.

Table 4.4: Severity mapping and gate decision

Raw severity	Tier	Gate decision
critical	Critical	Fail if any Critical finding
high, medium, moderate	Moderate	Review if no Critical and any Moderate
low, info, unknown	Warning	Pass if nothing above Warning

4.4.4 Reports, decisions, and the fix loop

Each report records the scan type, the gate decision, counts per tier, every finding with its location, CWE, root cause and recommendation, the dependency audit summary and the status of the model layer. It is saved in JSON and Markdown, can be exported as a PDF, and is pending until a person approves it. On approval, the reviewer chooses to proceed or to send the report to the Coder Agent. Sending it starts a Coder Agent revision whose request is the list of findings; when that revision finishes, the Security Agent runs again automatically and produces a new pending report. The loop continues until the reviewer proceeds, or the scan comes back clean. A reviewer can also mark individual findings as skipped, for example a known false positive, and ask questions about the report in a chat that has the report as its context.

4.4.5 Test discovery and generation

Figure 4.3 shows the QA pipeline. Discovery finds three kinds of target. Unit targets are TypeScript files under lib/ and models/ with at least one export. Integration targets are API route handlers together with the local model and library files they import, so that a test can exercise the

real path from request to handler to data. Regression targets are the feature's approved acceptance criteria, one test each. React pages and components are discovered too, but only so that they can be listed in the report as out of scope.

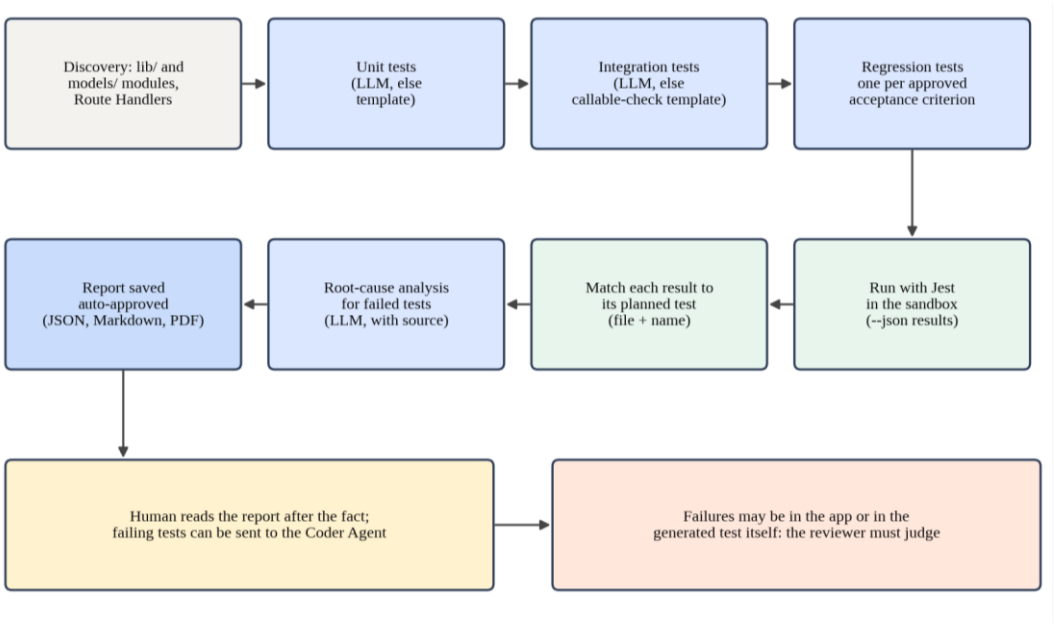


Figure 4.3: QA Agent pipeline

For each target the model receives the real source and returns two parts: JSON metadata describing each test case, its target, inputs and expected behavior, followed by a fixed marker and the test file itself. Splitting on a plain marker avoids the escape errors that come from putting a code file inside JSON. Tests that use skip, todo, each or concurrent are rejected, because they break the one-to-one link between planned and executed tests, and the attempt is retried with the reason, up to three times. If every attempt fails, a deterministic fallback writes a minimal test instead: that the module or handler can be loaded and called without crashing. Fallback tests are labelled as such in the report.

4.4.6 Execution, matching and analysis

Before the first run, the agent adds Jest and its Babel configuration to the project's development dependencies and installs them once. Tests run with Jest's JSON output in the sandbox. Each result is matched back to the test case that planned it by file and name, with a positional fallback within the same file when the model renamed a test. A planned test with no result is reported as failed with an explicit note, never as passed or silently dropped, and every result is either passed or failed.

If Jest produces no results at all for the planned tests, the report records an environment failure instead of a list of test failures. Otherwise, one batched model call analyses all failures together and adds a root cause and a recommendation to each.

4.5 Standards and Professional Practice

Findings are classified with CWE identifiers, so that each one names a recognised weakness class and links to public guidance. The combination of static analysis with model review follows the recommendation of Zhu et al. [3]and the three test categories follow common practice: unit tests for single modules, integration tests across modules, and regression tests tied to acceptance criteria so that later changes can be checked against the original requirement.

Dependency audits and test runs execute in a container, never on the host. Reports state what was not examined, which is ordinary professional practice for security and test reports, and the gate decision is presented as a recommendation for a person, not as an automatic verdict.

5 IMPLEMENTATIONS

5.1 Development Environment and Technology Stack

Both agents are Python packages inside the team's FastAPI backend. The code they examine is TypeScript, so parts of the work run in Node.js. Table 5.1 lists the main technologies.

Table 5.1: Technology stack used by the component

Concern	Technology	Why it was used
Agent backend	Python, FastAPI, Pydantic	Typed reports and streaming progress
Syntax-tree scan	TypeScript compiler API via a small Node script	A real parse of TS and TSX, including JSX attributes
Secret scan	Python regular expressions	Works on any text file, not only code
Dependency audit	npm audit in the Docker sandbox	Uses the project's own lock file; isolated from the host
Model layers	qwen3-coder via Ollama (provider service)	Local model; any configured provider can be used
Test framework	Jest with babel-jest	Structured per-test JSON results
Reports	JSON, Markdown, PDF builder	Machine-readable and human-readable forms of the same report
Frontend	React 19, Vite, Tailwind CSS 4	Report views, decision dialog and report chat
Testing	pytest	Fixture-based tests with recorded Node and model outputs

5.2 Implementation of the Core Modules

Table 2: Main modules of the component

Module	Lines	Responsibility
security_agent/agent.py	507	Standard and deep scan flows, report building, review layer, chat
security_agent/scanners.py	366	AST pattern scan, secret scan, dependency audit

security_agent/deep_scan.py	349	Source-reading scan in concurrent batches
security_agent/severity.py	60	Severity tiers, counts and gate decision
security_agent/prompt.py, schemas.py, pdf_builder.py	361	Prompts, report schema, PDF export
qa_agent/agent.py	621	Run flow, result matching, root-cause step, report, chat
qa_agent/generator.py	408	LLM generation with retries and deterministic fallbacks
qa_agent/executor.py	194	Jest setup, sandboxed run, result parsing
qa_agent/discovery.py	144	Unit, integration and out-of-scope discovery
qa_agent/prompt.py, schemas.py, pdf_builder.py	446	Prompts, report schema, PDF export

5.2.1 Security scanners

The pattern scan writes a short Node script that uses the TypeScript compiler to parse a file and emit an event for every call, constructor, JSX attribute and bracket property access, with its name and line. The Python side matches these events against the rule table. The script resolves TypeScript from the scanned project's own dependencies, which every generated project already has, rather than from the backend's. This step only parses code; it never executes it.

Testing the rule matching found a real bug. The rule for Math.random originally matched the name Math, but the syntax-tree script reports only the final part of a property call, random, so the rule could never fire. The rule now matches random, in the same way the other method rules match the method name. The tests for the scanner feed recorded events into the matcher, which is how this was caught without needing Node in the test environment.

The secret scan walks the project, skipping folders such as node_modules and .next, which in a built project contain tens of thousands of files, and applies the five rules line by line. The dependency scan runs npm audit with JSON output through the sandbox service. As implemented, the command includes the -offline flag, and the report's audit_ran_offline field is set from a fixed string rather than from the command actually run. Chapter 6 discusses what this means for the dependency results.

5.2.2 Model layers and reports

The review layer sends the model a numbered summary of the deterministic findings and parses its answer against a schema. Responses wrapped in Markdown fences are still parsed; malformed ones, missing fields and an unreachable provider all produce an empty list and a status note. The deep scan reads every scannable file, groups the files into batches under a character limit, and runs the batches concurrently with a bounded number of simultaneous calls; a failed batch is logged and contributes nothing. Concurrency was added after a scan of a multi-file project turned out to need ten to thirty sequential model calls.

The report builder merges the deterministic and model findings, computes the tier counts and gate decision, and saves the JSON and Markdown versions through the shared artefact service with a pending status. A streaming version of the deep scan reports progress batch by batch to the interface.

5.2.3 QA discovery, generation and execution

Discovery strips comments before looking for exports, so that a commented-out function is not treated as a test target and resolves imports such as @/models/Item to real files so that an integration test can be given the source of everything a route handler uses. Generated test files are written to a generated_tests folder in the workspace, which is cleared at the start of each run so that a test for a deleted module is never run again.

The executor's Jest configuration names the Babel configuration file explicitly. Without it, a live run showed babel-jest silently searching for its own configuration and failing to transform TypeScript. Jest writes its results to a JSON file, which the parser reads; Jest's own pending and todo statuses cannot occur, because the constructs that produce them are rejected at generation time.

5.2.4 Interface support

On the frontend, the component provides report views with colour-coded tier badges that also carry text labels, a decision dialog with the proceed and send-to-Coder options, controls to mark a finding as skipped, a button to start an AI deep scan, and a chat for each agent. Reviewers can also download a feature's code together with its QA report.

5.3 Integration with the Complete Solution

The component integrates with the Coder Agent in two directions. Forwards, it reads the same workspace the Coder Agent wrote and runs everything that executes through the same sandbox service. Backwards, the send-to-Coder action turns a security report into a Coder Agent revision, and the Coder Agent's verifier includes an informational check that every file named in the findings was touched by the fix. With the requirement stage, the QA Agent reads the approved acceptance criteria to write regression tests.

In the orchestrator, the security and QA nodes call the agents and advance without a graph interrupt. The approval service handles the security report's pending status like any other artefact, and the approval hook triggers the decision dialog.

5.4 Testing and Quality assurance

Table 5.3: Automated tests for the component

Area	Test files	Test functions
Security Agent: scanners, layers, reports, routes, skip	7	96
QA Agent: generation, execution, matching, reports, routes	10	99
Total	**17**	**195**

The tests concentrate on the parts that must never fail silently: the severity mapping for every raw word including unknown ones, the gate decision, every model-layer failure mode, the saving of the security report as pending rather than approved, the rejection of forbidden Jest constructs, the deterministic fallbacks, matching when the model renames a test, and the detection of an environment failure when Jest produces no results.

The last recorded run of the backend suite, on 31 August 2026, collected 239 test cases whose names refer to the security or QA stages. The failures recorded in that run's cache for this component all belong to tests that have since been renamed or removed, including stub tests from before the agents were implemented, so no current test is recorded as failing.

5.5 Deployment and Operation

The component needs Node.js on the backend machine for the syntax-tree scan and Docker for the sandbox. Jest is added to each generated project the first time the QA Agent runs and is shipped

with the project when it is downloaded. Reports are stored with the other artefacts and persist across restarts.

6 RESULTS AND EVALUATION

The chapter presents the findings according to the research questions. First, however, a summary of the results of the automated testing is provided. The evaluation is based on three sources, namely the controlled evaluation conducted by the team, the Login and Signup feature where vulnerability was intentionally planted.

6.1 Component-Level Test Results

Table 6.1: Key behaviors confirmed by automated tests

Behavior	**Test file**	**Result**
Every raw severity word, including unknown ones, maps to the right tier	security_agent	Pass
The gate decision follows the tier counts	security_agent	Pass
Malformed, fenced or missing model output, or an unreachable provider, never fails the scan	security_agent	Pass
The security report is saved as pending, not approved	security_agent	Pass
Each AST rule fires on its construct; duplicate events are reported once	security_ast_scanner	Pass
A failed deep-scan batch is skipped without aborting the scan	security_deep_scan	Pass
Findings can be marked and unmarked as skipped	security_finding_skip	Pass
Forbidden Jest constructs are rejected and fallbacks are used after failed attempts	qa_generator_fallback	Pass
Results match planned tests by file and name, with a positional fallback	qa_agent_matching	Pass
No Jest results for planned tests is reported as an environment failure	qa_environment_failure	Pass

6.2 Experimental Results

6.2.1 Detection of Known vulnerabilities (RQ1)

The controlled security evaluation determined a total of 7 true positives, 1 false positive, and 2 false negatives, achieving a precision of 87.5% and a recall of 77.8%. In order to assess how the different Security Agent layers contributed towards the detection, I reviewed the output of all reports issued while the five vulnerabilities presented in Table 3.2 were present in the Login and Signup feature. This included three normal scans and four deep AI scans. The data related to the detected vulnerabilities by each layer is shown in Table 6.2.

Table 6.2: Detection of the seeded vulnerabilities by layer

Seeded vulnerability	**Deterministic layers (7 reports)**	**Review layer (3 standard scans)**	**AI deep scan (4 reports)**
V1 Hardcoded credential (CWE-798)	7 of 7	Not as a new finding	4 of 4
V2 Raw HTML rendering (CWE-79)	7 of 7	Not as a new finding	4 of 4
V3 NoSQL injection (CWE-943)	0 of 7	0 of 3	4 of 4
V4 Weak bcrypt cost (CWE-916)	0 of 7	0 of 3	4 of 4
V5 Password in log (CWE-532)	0 of 7	0 of 3	4 of 4
False or mislabelled findings	None	1 per report (3 in total)	1 in 3 of the 4 reports

The deterministic layers behaved exactly as designed. They found the two vulnerabilities that match a rule, the hardcoded connection string and the raw HTML attribute, in every report, at the right file and line, with no false findings. They did not find the other three, because none of them is a single dangerous construct: an injection depends on how a request value reaches a query, a weak hash on the value of an argument, and a leak on what is being logged.

The review layer, which sees only a summary of the deterministic findings, added one finding to each standard scan, and none of the three was useful. Each pointed at a location the deterministic layers had already reported, but with the wrong vulnerability class: the raw HTML finding was relabelled as an insecure direct object reference, and in two later reports a finding was labelled as SQL injection (CWE-89), first for the hardcoded connection string and then for the raw HTML attribute. The project does not use SQL at all.

The AI deep scan, which reads the source, found all five vulnerabilities in all four reports, with correct files and CWE classes. It also reported, in three of the four, that passwords were stored in plain text. They were not: the code hashed them with bcrypt, at the deliberately weak cost of 1, which the scan had already reported correctly as V4. For RQ1, then, the deterministic layers are precise but narrow, the deep scan has much wider reach at the cost of occasional false findings, and the summary review layer added noise rather than detection.

The dependency layer reported no vulnerable packages in any stored report. Because the audit command runs with the -offline flag, npm may not have been able to consult the advisory data at all, so these zero results cannot be read as evidence that the dependencies are safe.

6.2.2 Consistency between scans (RQ2)

Figure 6.1 shows the findings in all nineteen security reports of the Login and Signup feature, split by the layer that produced them. From the project's git history, reports 2 to 8 scanned the same seeded code, reports 9 to 12 the code after the first fix, and reports 13 to 19 the same code after the second fix.

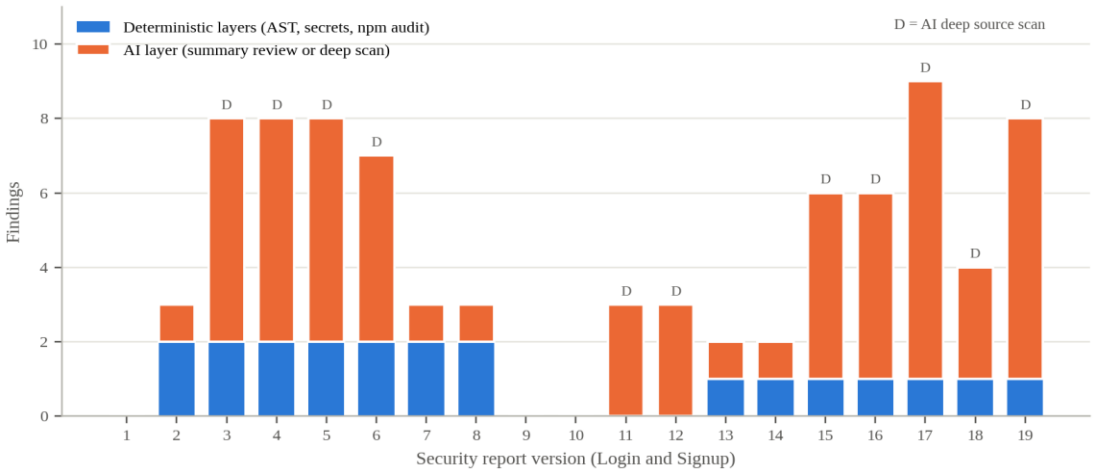


Figure 3.1: Findings per security report for the Login and Signup feature, by producing layer

On the seeded code, the deep scan was consistent about what mattered: the five seeded vulnerabilities appeared in all four reports. It was less consistent about the rest. The false plaintext-password finding came and went, and the weak-hashing finding was Critical in three reports and Moderate in the fourth. On the fixed code, five deep scans of identical code reported 6, 6, 9, 4 and 8 findings, one of them with a Critical finding and four without. The plaintext-password claim reappeared in two of them, once at high severity and once at critical, although the fixed code hashes passwords with a cost factor of 12. The standard scans of the same code, by contrast, reported the same two findings each time.

The retail store feature shows the same pattern on a smaller scale. Two deep scans run four minutes apart on the same code returned five and six findings that barely overlapped. Some were reasonable, such as an image URL accepted without validation, which matters because the Coder Agent had chosen to accept image URLs. Others were not, such as a claim of insecure deserialization simply because the code uses Mongoose, a "hardcoded secret" for reading an environment variable, and four copies of a logging finding filed under CWE-319, which concerns cleartext transmission. For RQ2, the deterministic layers are fully repeatable, and the deep scan is not: a reviewer should treat a single deep-scan report as a list of leads, not as a measurement.

6.2.3 Evidence from generated tests (RQ3)

The QA assessment showed the results of 2 tests performed for the assessment of the Item Listing, 14 tests out of 16 passed for Authentication, and 18 tests out of 20 were satisfactory for Shopping Cart. The enhanced evaluation comprises a total of 60 tests out of which 54 were a success, giving an overall rate of 79%. The QA agent cannot measure the level of the code coverage; hence all the figures are derived from the given assessment and not through the agent's analysis.

Eight QA reports kept throughout the development projects do not provide the best outcome, and hence they should also be considered. Table 6.3 gives a summary of the 46 trials in those reports.

Table 6.3: Outcome of all test cases in the stored QA reports

Generated by	**Passed**	**Failed**	**Not run (early versions)**
Model (LLM)	0	15	22

Deterministic fallback	5	0	4
Total	**5**	**15**	**26**

No model-generated test in the stored reports passed, and in every case, I examined the failure was in the test, not in the feature. In the retail store report, all five regression tests failed with "POST is not defined" or "GET is not defined": they called the route handlers without importing them. The same error appears in the last Login and Signup report. In the first Login and Signup report, six-unit tests failed because they read a property of the Mongoose schema that does not exist. The 26 tests marked as not run came from the earlier reports, in which test files that failed to load produced no results for their planned tests; the agent then reported such tests as skipped, and it now reports them as failed. The five passing tests were all minimal fallback checks that a module can be loaded and called.

Two parts of the design did work well. The root-cause analysis correctly identified the missing import in every regression failure, so a reviewer reading the report would see that the tests, not the feature, were at fault. And one failing unit test in the first Login and Signup report did reveal a real problem: it expected the connection helper to use the test connection string, and received the hardcoded fallback credential, which was the seeded vulnerability V1. For RQ3, the execution, matching and analysis machinery produces honest results, but in the stored runs the model-generated tests were not good enough to serve as evidence about the feature. The difference from the paper's results remains to be explained, and I return it in Chapter 7.

6.2.4 The fix loop (RQ4)

The Login and Signup feature went through the full loop twice. The standard scans of the seeded code reported V1 and V2, and the reviewer sent them to the Coder Agent. The resulting revision removed both, and the next two standard scans passed with no findings. The deep scans run immediately afterwards still reported V3, V4 and V5, which the standard scan cannot see, and a second revision, whose plan states that it addresses the injection in the login route, removed them: the fixed code restricts the filter to an allow-list, uses a bcrypt cost factor of 12 and no longer logs the password.

The fix itself produced a new finding. The allow-list loop indexes the filter object by a variable named key, which matches the dynamic property access rule, so every later standard scan reported a Moderate finding on exactly the code that fixed the injection. The deep scans also kept reporting a NoSQL injection risk in the same route. That is arguably still valid: the allow-list admits the email field, so a caller can still pass an object with a query operator in place of an email string. For RQ4, the loop did remove all five seeded vulnerabilities in their original form within two revisions, but the reports that followed needed a person to tell a residual risk from a false alarm.

6.3 Stakeholder and User Validation

No security professionals, testers or businesses outside the project team used the component. Reviews were made within the team, by people who knew which vulnerabilities had been seeded. A reviewer who did not know would face the harder task of judging deep-scan findings on their own, which makes the inconsistency in Section 6.2.2 more significant.

Team use did shape the design. The proceed or send-to-Coder choice, the AI deep scan, concurrent batching, the explanation of why each test failed, and the rule that every test is reported as passed or failed were added at team members' request while using the agents on real features.

6.4 Integration and Performance Results

Both agents ran in every pipeline run of the team's evaluation. The Security Agent reported 0, 1 and 1 issues for Item Listing, Authentication and Shopping Cart. With human gates switched off, the team recorded 4 security issues and 5 escaped defects, against 1 and 1 with gates on. Table 6.4 shows the cost of each run; the logs do not separate the time of each stage.

Table 6.4: Execution performance of the full pipeline

Run	Total time (min)	LLM calls	Retries	Tokens
Item Listing	36.4	24	4	31,800
Authentication	52.7	31	6	42,600
Shopping Cart	56.8	34	7	47,900
Baseline	31.2	18	3	25,400

The stored reports give a rough idea of the security stage's own cost. A standard scan and a deep scan of the same code were typically a few minutes apart, and consecutive deep scans of the retail store feature were four minutes apart, so a scan takes minutes rather than tens of minutes. The QA stage is slower, because its first run installs Jest and every run execute the suite in the sandbox.

6.5 Summary of Findings

Table 6.5: Summary of findings by research question

RQ	Finding	Strength of evidence
RQ1	Deterministic layers found 2 of 5 seeded issues every time with no false findings; the deep scan found 5 of 5 every time, with a false finding in 3 of 4 reports; the review layer added only mislabelled duplicates	One seeded feature; paper's controlled set
RQ2	Deterministic results were identical across scans; deep-scan results on identical code varied from 4 to 9 findings	Nineteen reports; git history
RQ3	Execution and failure analysis were honest, but no model-generated test passed in the stored runs, all because of test defects	Eight stored reports; paper's case study
RQ4	Two fix rounds removed all five seeded issues; the fix triggered a new rule finding and left a partly valid injection risk	One feature's full loop

7 DISCUSIONS

7.1 Interpretation of the Results

The security results support the hybrid design, but not in the way I expected at the start. I expected the model layer to be a modest addition to a strong deterministic core. In practice, the deterministic core was perfectly reliable but narrow, finding two of the five seeded vulnerabilities, while the source-reading deep scan found all five every time. On detection alone, the deep scan was the more useful layer.

Its weakness was consistency. The same code produced between four and nine findings depending on the run, a plainly false finding came and went, and severities shifted. A reviewer who saw one report could not tell which findings were stable. This is precisely where the deterministic layers earn their place: they give the reviewer findings that do not change, can be traced to a rule, and are never invented. The two kinds of layer answer different needs, which is the point Zhu et al. [3] make about combining them.

The summary review layer, on the other hand, did not justify itself in these runs. Shown only a summary, it could do little but restate the deterministic findings, and it did so with the wrong vulnerability class. A model given less context produced worse output, which argues for giving any model layer the real code or nothing.

The QA results are the most sobering part of this work. The machinery around the tests, execution in the sandbox, matching, honest statuses and root-cause analysis, worked as designed and told the reviewer the truth: that the tests were broken. But the tests themselves, in the stored runs, never provided passing evidence about a feature. The most common defect, calling a route handler without importing it, is simple and repeated, which suggests a fixable problem in the regression prompt or a missing deterministic check rather than a fundamental limit.

The gap between these stored runs and the QA results reported in our paper needs to be stated plainly. The paper's results come from the team's case study on a different project, and some of its figures, such as code coverage, are not produced by the agent as implemented. I cannot reconcile the two from the stored artefacts alone. The responsible reading is that the QA Agent's test quality varies strongly between runs, and that the paper's QA figures should be repeated with the current implementation before they are relied on.

7.2 Comparison with Related Work

Zhu et al. [3]recommend combining static analysis with LLM review; my results give a small, concrete picture of what each half contributes and costs. Wang et al. [4] warn that generated tests often fail to run or check the wrong thing; the stored QA reports are a direct example, and the root-cause step shows one way to keep such failures from misleading a reviewer. AgentCoder [7] puts test execution inside the generation loop, so that a broken test is fixed before anyone sees it; the QA Agent retries only for forbidden constructs, not for tests that fail because of their own errors, which is the most obvious lesson to take from it.

Compared with ChatDev [1] and MetaGPT [2], whose reviewer and QA roles are agents talking to agents, my component reports to a person and lets that person send findings back for a fix. The fix loop in Section 6.2.4 is consistent with the value Takerngsaksiri et al. [10] found in keeping engineers involved in an agent's work: the loop removed every seeded vulnerability, but a person needed to judge the findings that followed.

7.3 Implication for Practice

Four practical points follow. First, evaluate a security scanner on code with known vulnerabilities; the clean scans in this project said nothing about detection, while the seeded feature said a great deal. Second, if a model reviews code, show it the code. Third, show the reviewer which findings are deterministic and which come from a model, and treat a single model report as a list of leads. Fourth, a test report should separate "the test is broken" from "the code is broken", as the root-cause step does, and generated tests should be repaired against their own errors before they are reported.

7.4 Threats to Validity

Internal validity. The seeded vulnerabilities were introduced and judged by the team that built the agents, and the reviewers knew where they were. The classification of extra findings as false or valid was done by one person and could be disputed, for example on the residual injection risk.

External validity. The seeded set has five vulnerabilities in one small feature on one stack, and most runs use one model. Real applications have more code, more subtle flaws and real dependencies, and a different model may behave very differently.

Construct validity. Counting detections of seeded vulnerabilities measures recall on that set, not overall security. Counting passing tests measures the tests as much as the code, as Section 6.2.3 shows.

7.5 Limitations

- The dependency audit runs with -offline, so its zero results may mean the audit could not run, and the report's audit_ran_offline field is set from a fixed string rather than from the command.
- The deterministic rule set is small and finds only single-construct vulnerabilities; one rule fires on the allow-list pattern it recommends.
- Deep-scan results vary between runs on identical code, including false and shifting-severity findings.
- No model-generated test passed in the stored runs; the regression prompt repeatedly omitted imports, and the agent does not retry tests that fail because of their own errors.
- The QA Agent does not measure code coverage and does not test React pages or components.
- The stages dealing with Security and QA operate without an interruption of the graph. Even though Security report necessitates an approval from a human at the artefact level, the QA report undergoes an approval phase and is then saved as approved. Currently, the output from Security report guides the reviewer instead of blocking the pipeline.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value of the Component

The team's common report states that teams employing artificial intelligence to develop software should know what their programming produced and accept its output. The verification step is the stage at which this becomes a reality, since the client or manager will ask two questions about any of the features generated: is it secure and has it been tested? Meanwhile, the developers have begun to doubt the validity of the AI-created code. According to the results of the Stack Overflow Developer Survey conducted in 2025, merely 32.7% of software creators trust the validity of AI-generated code, while 45.2% pointed out that debug of AI-generated code takes very long [15]. A tool that creates the code unnoticed increases the time required for this work, while a tool that checks the code and documents its evidence minimizes it.

My component answers both questions with evidence that a person can inspect findings tied to files, lines and CWE classes, and tests that were run. It also turns a finding into a fix, by sending it to the Coder Agent as a revision instead of leaving it in a report. The evaluation shows each part of this value. On the team's-controlled vulnerability set the Security Agent reached a precision of 87.5% and a recall of 77.8%. On the seeded Login and Signup feature, the deterministic layers found their two vulnerabilities in every report with no false findings, the AI deep scan found all five in every report (Table 6.2), and two rounds of the fix loop removed all five (Section 6.2.4). On the QA side, the root-cause analysis correctly explained every failed regression test, and one failing test exposed the seeded hardcoded credential (Section 6.2.3).

Table 8.1 shows who would use the component and what each group gains.

Table 8.1: Target users of the component

User group	Need	What the component provides
Developers and leads in small teams without a security specialist	Know whether generated code is safe before merging it	Deterministic scans, an AI deep scan and dependency checks, mapped to Critical, Moderate and Warning tiers

Teams without a dedicated tester	Evidence that a feature works	Unit, integration and regression tests generated, run in the sandbox and explained when they fail
Agencies delivering to clients	Proof of quality to hand over with the work	Security and QA reports in Markdown and PDF, with the reviewer's decision recorded
Store owners	Protect customer accounts, passwords and orders	Findings such as exposed credentials, injection and weak hashing caught and fixed before release

8.2 Market competitive position

The first market is the one AutoForge targets: small e-commerce development teams and agencies, beginning in Sri Lanka, whose IT and business process management industry has more than 680 companies and over 700 active startups [16]. Small firms in this market rarely employ a security specialist or a dedicated tester, which is the gap this component fills.

Security and testing tools are a mature market, and the leading products are inexpensive for small teams. Table 8.2 compares them with this component, using published prices in October 2026 converted at 330.43 rupees to the dollar [21].

Table 8.2: Alternatives for checking generated code

Alternative	**Typical cost**	**Strength**	**Weakness compared with this component**
Manual review and testing	Specialist time	Expert judgement	Slow; often skipped in small teams
GitHub Secret Protection and Code Security	USD 19 + 30 (about LKR 16,200) per active committer per month [18]	Mature secret scanning and semantic code analysis	Not linked to the generating agent; no generated tests; code must be hosted on GitHub
Snyk	Free tier; Team from USD 25 a month for up to 10 developers [19]	Mature code and dependency scanning	No link to an approved design; no tests; no human gate in a generation pipeline

SonarQube Cloud	Free up to 50k lines; Team from USD 34 a month [20]	Repeatable rule-based code quality and security	Rules only; no model reach; no tests or fix loop
Agent frameworks with QA roles, e.g. MetaGPT [2], AgentCoder [7]	Free (open source) plus model fees	Testing inside the pipeline	Agents review agents; no person decides; no security stage
This component (within AutoForge)	Included from the free Community edition	Deterministic core, model reach, executed tests, fix loop, human decision	Small rule set; noisy model layer; generated tests not yet reliable

8.3 Business Model and Feasibility

The component fits within the open-core model and the packages proposed in the common report (Table 7.5): a free self-hosted Community edition, Team seats at LKR 6,000 per month, Agency seats at LKR 10,000 per month, and enterprise and academic licenses. Security and QA are included in every package, because a pipeline that generates unchecked code would undermine the whole product. Table 8.3 shows the paid elements that depend directly on my work.

Table 8.3: Revenue elements that depend on this component

Revenue element	Price	What this component contributes
Client-ready reports (Agency package)	LKR 10,000 per seat per month	Security and QA reports already export to PDF; the Agency package adds client branding and a summary page for hand-over
Curated security rule packs	Proposed at LKR 30,000 per pack, like knowledge packs	The rule tables are small and separately tested, so rules for a new framework can be added as a pack
Hosted deep scans and test runs	Part of each hosted feature run; extra runs LKR 400	The AI deep scan and Jest runs execute on the hosted service for teams without capable hardware

Enterprise / on-premises license	From LKR 1,500,000 per year	All scans and tests run inside the customer's network; no code leaves it

The cost of running the component is modest and predictable. A security scan took minutes rather than tens of minutes in the stored reports (Section 6.4). The deep scan sends the source to the model in batches of up to 12,000 characters, so its cost grows with the size of the code; a hosted version should therefore scan only the files a feature changed or count larger scans as more than one run. With the local model, all of this costs nothing in API fees, and a whole hosted feature run costs at most about LKR 160 in model fees.

The results in Chapter 6 also say clearly what must be fixed before the component can be sold. Table 8.4 lists the changes. A security product cannot ship with a check that may silently not have run, or with tests that fail because of their own mistakes.

Table 8.4: Work needed before a commercial release

Issue found	**Evidence**	**Change needed**
The dependency audit ran offline and may not have checked anything	Section 6.2.1	Run the audit online and state in the report whether advisory data was consulted
No model-generated test passed in the stored runs, all because of test defects	Table 6.3	Run and repair generated tests before reporting; check that every handler a test calls is imported
Deep-scan results on identical code varied from 4 to 9 findings	Section 6.2.2	Run the deep scan more than once and mark only recurring findings as confident
The review layer added only mislabeled duplicates	Section 6.2.1	Remove it or give it the source code, as the deep scan has
The fix for an injection triggered a new rule finding	Section 6.2.4	Teach the dynamic property rule to recognize an allow-list check
Some stored reports quote credentials found in the code	Section 3.2	Mask every secret in reports before they are saved or shared

8.4 Sustainability and Maintenance

Technically, the rule tables, secret patterns and severity mapping are small, separately tested and easy to extend, and the model layers are isolated so that a better model can be adopted by configuration. The dependencies most likely to cause maintenance work are the TypeScript compiler, whose syntax-tree node names the scan relies on, and Jest with Babel, whose configuration already caused one silent failure.

Socially and ethically, the component must not create false confidence. A clean report, especially from the deep scan, can be read as "secure", which no automated tool can promise. The reports therefore present the gate decision as advice, state what was not checked, and label model findings. Used this way, the component can also help developers learn, since every finding comes with a root cause, a CWE class and a recommendation.

8.5 Contribution to the Sustainability Development Goals

The common report links AutoForge to SDG 9 (Industry, Innovation and Infrastructure) and SDG 8 (Decent Work and Economic Growth). My component's contribution to SDG 9 is safer AI-assisted software development for small firms, which can run security checks and tests locally without specialist staff. Its contribution to SDG 8 would be fewer security defects reaching small online businesses and their customers. Both are potential; a pilot measuring defects found before and after release would be needed to show them.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis sets out to give each generated feature security findings and test results that a reviewer can trust and act on. The Security Agent combines a syntax-tree scan, a secret scan and a dependency audit with a model layer, maps everything to three fixed severity tiers, and lets the reviewer send findings back to the Coder Agent for a fix. The QA Agent discovers what can be tested, generates and runs unit, integration and regression tests in the sandbox, and explains failures.

The evaluation showed a clear division of labour. The deterministic layers were precise and repeatable but narrow; the source-reading deep scan found every seeded vulnerability but was inconsistent and sometimes wrong; and the fix loop removed all seeded vulnerabilities within two revisions, with a person still needed to judge what followed. The QA machinery reported honestly, but the model-generated tests in the stored runs never passed, because of their own defects. The main lesson is that verification by AI needs a deterministic anchor, repeated runs and a person, and that generated tests must themselves be verified before they count as evidence.

9.2 Achievement of Objectives

Objective	Outcome	Status
O1: Security Agent with deterministic layers, model layer and severity tiers	Seven AST rules, five secret rules, npm audit, review and deep-scan layers, three tiers and gate decision; audit runs offline	Largely achieved
O2: QA Agent generating, running and explaining tests	Discovery, three test categories, fallbacks, Jest in the sandbox, matching, root-cause analysis; generated tests unreliable	Partly achieved
O3: Human decision and fix loop with the Coder Agent	Proceed or send-to-Coder decision with automatic re-scan; removed all seeded issues in two rounds	Achieved
O4: Evaluation including known vulnerabilities	195 tests, seeded feature, 22 security and 8 QA reports, case study	Achieved at limited scale

9.3 Contribution

The main contribution is a tested design for the verification end of a human-gated agent pipeline, in which deterministic security analysis and model-based review are combined, every finding is mapped to one severity scale, tests are actually executed, and findings flow back to the coding agent at a person's decision. A second contribution is evidence, from a seeded feature and stored reports, of how the layers differ in reach, precision and consistency. A third is an honest account of the failure modes of generated tests and of the places where the implementation and our paper's description have diverged.

9.4 Future Directions

The most direct improvements follow from Chapter 6. The dependency audit should run online and record whether it consulted advisory data. Generated tests should be run first and repaired when they fail for their own reasons, for example a missing import, before their results are reported; a simple deterministic check that every referenced handler is imported would have caught most stored failures. The dynamic property access rule should recognize a check against an allow list.

Beyond these, the deep scan could be run several times and only recurring findings shown as confident, with the rest marked as tentative. Its findings could be cross-checked against the deterministic layers and the code, for example by rejecting a plaintext-password claim when the code calls a hashing function. The QA Agent should measure coverage itself and add browser-based tests for pages. Finally, the evaluation should use a larger seeded set across several features and models, reviewed by people who do not know where the vulnerabilities are.

REFERENCES

[1] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, *et al.*, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, 2024, pp. 15174–15186.

[2] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, S. Yau, Z. Lin, L. Zhou, *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024, pp. 23247–23275.

[3] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, "When software security meets large language models: A survey," *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 2, pp. 317–334, 2025.

[4] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Trans. Softw. Eng.*, vol. 50, no. 4, pp. 911–936, 2024.

[5] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 50528–50652.

[6] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, *et al.*, "OpenHands: An open platform for AI software developers as generalist agents," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025, pp. 65882–65919.

[7] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv preprint* arXiv:2312.13010, 2023.

[8] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, 2024.

[9] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–30, 2025.

[10] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-in-the-loop software development agents," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, 2025, pp. 342–352.

[11] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[12] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024, pp. 54107–54157.

[13] H. N. Phan, T. N. Nguyen, P. X. Nguyen, and N. D. Bui, "HyperAgent: Generalist software engineering agents to solve coding tasks at scale," *arXiv preprint* arXiv:2409.16299, 2024.

[14] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," *arXiv preprint* arXiv:2407.01489, 2024.

[15] Stack Overflow, "2025 Developer Survey: AI," 2025. [Online]. Available: https://survey.stackoverflow.co/2025/ai (accessed Oct. 8, 2026).

[16] "SLASSCOM reflects on a year of creating opportunities, strengthening Sri Lanka's knowledge and innovation industry," Daily FT, Jul. 15, 2026. [Online]. Available: https://www.ft.lk/it-telecom-tech/SLASSCOM-reflects-on-a-year-of-creating-opportunities-strengthening-Sri-Lanka-s-knowledge-and-innovation-industry/50-794672 (accessed Oct. 8, 2026).

[17] "Data protection compliance regime takes effect on 1 Jan. 2027," Daily FT, 2026. [Online]. Available: https://www.ft.lk/front-page/Data-protection-compliance-regime-takes-effect-on-1-Jan-2027/44-795776 (accessed Oct. 8, 2026).

[18] GitHub, "GitHub security plans." [Online]. Available: https://github.com/security/plans (accessed Oct. 8, 2026).

[19] Snyk, "Plans and pricing." [Online]. Available: https://snyk.io/plans/ (accessed Oct. 8, 2026).

[20] SonarSource, "Plans and pricing." [Online]. Available: https://www.sonarsource.com/plans-and-pricing/ (accessed Oct. 8, 2026).

[21] Exchange-Rates.org, "USD to LKR: Convert US Dollars to Sri Lanka Rupees." [Online]. Available: https://www.exchange-rates.org/converter/usd-lkr (accessed Oct. 7, 2026).

GLOSSARY

Abstract syntax tree	A tree representation of source code produced by a parser, used to find real code constructs.
CWE	Common Weakness Enumeration, a public catalogue of software weakness types.
Deep scan	The model layer that reads the real source code in batches to find vulnerabilities.
Deterministic layer	A scan whose result depends only on fixed rules and the code, not on a model.
Environment failure	A QA run in which the test framework produced no results at all for the planned tests.
False positive	A reported finding that is not a real problem.
Fix loop	Sending security findings to the Coder Agent as a revision, then re-scanning the result.
Gate decision	The pass, review or fail recommendation derived from the severity counts.
Integration test	A test that exercises a route handler together with the modules it imports.
Regression test	A test written from an approved acceptance criterion, to be re-run after later changes.
Review layer	The model layer that reviews a summary of the deterministic findings.
Root-cause analysis	A model step that explains why each failed test failed.
Seeded vulnerability	A known flaw was introduced deliberately to measure whether a scanner detects it.

Severity tier	One of Critical, Moderate or Warning, the single scale all findings are mapped to.
Unit test	A test of one module in isolation.

APPENDICES

Appendix A: Supporting Technical Material

A.1 Deterministic security rules

Table A.1: Pattern and secret rules

Rule	Detects	Severity	CWE
SEC-JS-001	eval() call	Critical	CWE-95
SEC-JS-002	new Function(...)	High	CWE-95
SEC-JS-003	child_process exec / execSync	High	CWE-78
SEC-JS-004	document.write()	Medium	CWE-79
SEC-JS-005	dangerouslySetInnerHTML attribute	High	CWE-79
SEC-JS-006	obj[sort|order|field|key|column] access	Medium	CWE-915
SEC-JS-007	Math.random() call	Low	CWE-338
SEC-SECRET-AWS-KEY	AWS access key ID	Critical	CWE-798
SEC-SECRET-GENERIC-KEY	API key, secret key or access token literal	Critical	CWE-798
SEC-SECRET-PRIVATE-KEY	Private key block	Critical	CWE-798
SEC-SECRET-MONGO-URI	MongoDB URI with an embedded password	Critical	CWE-798
SEC-SECRET-JWT	Literal JSON Web Token	Critical	CWE-798

A.2 Test catalogue

Table A.2: Test files for the component

Test file	Functions	What it covers
test_security_agent	27	Tiers, gate, model-layer parsing and failures, pending status
test_security_agent_routes	20	Scan, deep scan, decision and chat endpoints
test_security_ast_scanner	9	Rule matching on recorded syntax-tree events
test_security_deep_scan	17	Batching, concurrency, failed batches, IDs
test_security_finding_skip	8	Marking findings as skipped
test_security_scanners	5	Secret and dependency scans
test_security_pdf_builder	10	Security report PDF
test_qa_generator_fallback	27	Generation retries, forbidden constructs, fallbacks
test_qa_agent_matching	14	Result matching and Markdown report
test_qa_agent_routes	13	QA endpoints
test_qa_pdf_builder	11	QA report PDF
test_qa_jest_parser	8	Jest JSON parsing
test_qa_root_cause_analysis, test_qa_run_stream	12	Failure analysis; streamed runs
test_qa_environment_failure, test_qa_executor	10	Environment failures; Jest setup and runs
test_qa_agent_chat	4	Report chat
Total	**195**	

Appendix B: Disclosure of AI Tool Use

In line with the module policy, this appendix records how generative AI tools were used in this work.

Table B.1: Use of generative AI tools

Activity	**Tool**	**How it was used**	**How the output was checked**
System runtime	qwen3-coder via Ollama	The model inside the Security and QA agents, as described in this thesis	Deterministic layers, tests, seeded evaluation and human review
Figures	Claude (Anthropic)	Drawing the design diagrams and the findings chart from the stored reports	Checked against the implementation and reports by the author